Arquitetura MVC para Desenvolvimento de APIs

Arquitetura MVC para Desenvolvimento de APIs

Área Tecnológica: DESENVOLVIMENTO DE SISTEMAS

Público-Alvo: Ensino Técnico

SUMÁRIO

Capítulo 1: Fundamentos da Arquitetura MVC: Organizando o Caos 5

PARA INICIAR *5*

DESAFIO *5*

ESTUDO E PRÁTICA *6*

1.1 O Núcleo Conceitual: Desvendando o MVC 6

1.2 O Benefício Central: Separação de Preocupações 9

MÃO NA MASSA *10*

Passo 1: Estruturando as Pastas do Projeto 10

Passo 2: O Fluxo na Prática - Listando Produtos 11

RESUMINDO *13*

PARA CONCLUIR *14*

Capítulo 2: Refatoração para MVC e Design Patterns na Prática 15

DESAFIO 15

ESTUDO E PRÁTICA 16

2.1. A Arquitetura MVC como Pilar da Organização *16*

2.2. Indo Além: Padrões de Projeto para um Código Blindado *18*

MÃO NA MASSA 20

Passo 1: Criar a Estrutura de Diretórios *21*

Passo 2: Criar a Rota e o Controller *21*

Passo 3: Criar o Repositório *22*

Passo 4: Criar o Serviço (O Cérebro da Operação)..... *23*

RESUMINDO 24

PARA CONCLUIR..... 25

Capítulo 1: Fundamentos da Arquitetura MVC: Organizando o Caos

Olá! Seja muito bem-vindo ao nosso estudo sobre Arquitetura de Software.

PARA INICIAR

Olá! Você construiu uma API funcional na SA1, um grande passo e um marco importante na sua jornada como desenvolvedor. Parabéns por isso! Mas, como em todo projeto de sucesso, o que acontece quando ele começa a crescer? Novos desenvolvedores chegam à equipe, novas funcionalidades são solicitadas pelos clientes, e a complexidade aumenta. O código que antes era simples, claro e direto pode rapidamente se transformar em um labirinto confuso e de difícil manutenção.

Nesta unidade, vamos mergulhar em um conceito fundamental para qualquer profissional de desenvolvimento que busca construir sistemas robustos e escaláveis: a **arquitetura de software**. Descobriremos como a organização arquitetural, especificamente o padrão **MVC (Model-View-Controller)**, pode ser o mapa para o sucesso e a longevidade do nosso projeto "SaborDigital", transformando o caos potencial em uma estrutura ordenada e profissional. Ao final desta jornada, você terá uma compreensão profunda de como estruturar suas aplicações de forma limpa, eficiente e preparada para o futuro.

DESAFIO

Confira, a seguir, uma situação prática relacionada aos conteúdos que você vai estudar neste capítulo.

A equipe da TechFood está muito animada com o sucesso inicial da API "SaborDigital". No entanto, um problema começou a surgir. Ao receber a tarefa de adicionar um simples campo de `categoria` aos produtos do cardápio, um desenvolvedor júnior levou quase um dia inteiro para concluir a tarefa. O motivo? A lógica de negócio, o acesso ao banco de dados e o tratamento da requisição HTTP estavam todos misturados em um único e longo arquivo. A confusão para encontrar o local certo da alteração foi tão grande que, ao final, a implementação gerou um bug que quebrou a listagem de todos os produtos na aplicação.

Diante desse cenário, a liderança técnica se pergunta: **Como podemos evitar que isso aconteça de novo e preparar nosso código para funcionalidades mais complexas, como o sistema de upload de imagens de produtos que será solicitado na SA2?** A resposta está na organização que vamos aprender a seguir.

ESTUDO E PRÁTICA

Durante nossos estudos, vamos explorar a fundo o padrão arquitetural que é um pilar no desenvolvimento de aplicações web e APIs em todo o mundo.

1.1 O Núcleo Conceitual: Desvendando o MVC

Imagine tentar construir um prédio sem uma planta arquitetônica. Cada equipe (elétrica, hidráulica, estrutural) trabalharia de forma isolada e descoordenada, resultando em um desastre. No software, acontece o mesmo. Uma arquitetura é a "planta baixa" do nosso código, definindo onde cada peça de lógica deve residir.

O MVC (Model-View-Controller) é um dos padrões de arquitetura mais consagrados e utilizados no mercado. Sua proposta é simples e poderosa: dividir a aplicação em três componentes principais, cada um com uma responsabilidade bem definida.

#PraCegoVer: Diagrama técnico isométrico, estilo vetorial flat. Fundo branco limpo. Representação do fluxo de dados na arquitetura MVC. Três blocos principais com ícones universais: um bloco 'Controller' (ícone de engrenagem de processamento), um bloco 'Model' (ícone de cilindro de banco de dados), e um bloco 'View' (ícone de documento JSON `{}`). Setas azuis nítidas mostram o fluxo: 1) Requisição HTTP entra no Controller. 2) Controller invoca métodos do Model. 3) Model retorna dados ao Controller. 4) Controller formata a resposta usando a View. 5) View gera a Resposta HTTP para o cliente. Rótulos de uma única palavra: 'Requisição', 'Controller', 'Model', 'View', 'Resposta'. Cores corporativas: azul marinho, cinza e branco. High contrast, design limpo e profissional.

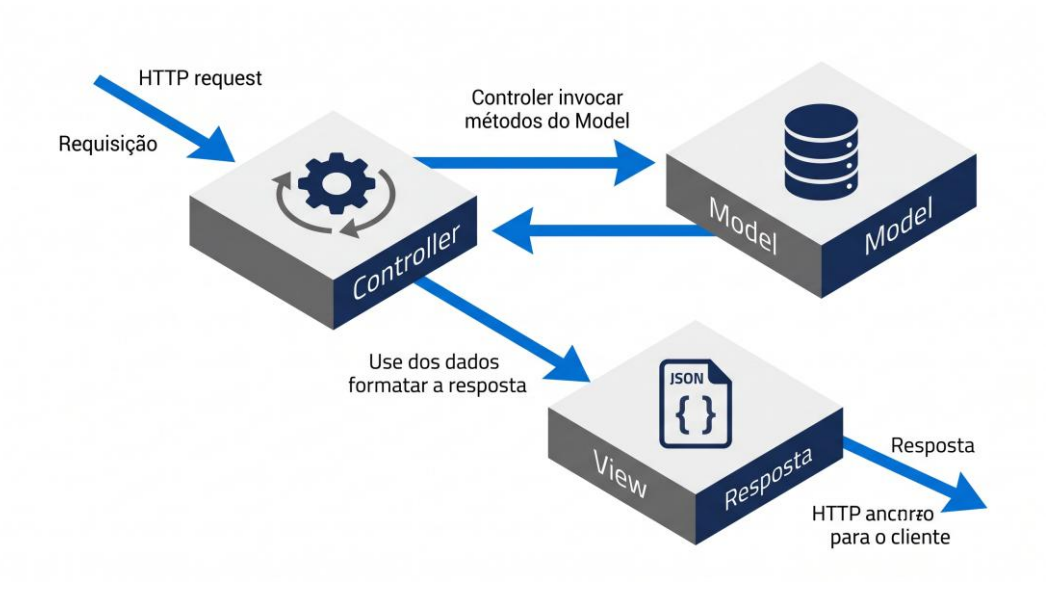

Figura: Gerada por AI

O Model (Modelo): O Especialista em Dados

O Model é o **cérebro** da nossa aplicação. Sua única preocupação são os dados e as regras que os governam (regras de negócio). Ele é responsável por:

Comunicar-se com o banco de dados (ler, escrever, atualizar, deletar).

Validar dados (Ex: um campo de e-mail deve ter um formato de e-mail válido).

Implementar regras de negócio complexas (Ex: ao criar um pedido, verificar se há estoque do produto).

Analogia: Pense no Model como o Chefe de Cozinha do "SaborDigital". Ele é o único que conhece as receitas (regras de negócio) e sabe exatamente onde cada ingrediente está no estoque (banco de dados). Você não pergunta ao garçom como se prepara o prato; você pergunta ao chefe.

A View (Visão): A Apresentação dos Dados

A View é a camada responsável por **apresentar os dados** ao usuário final. Em uma aplicação web tradicional, a View seria o código HTML e CSS. No nosso contexto de APIs REST, a "aparência" é, na verdade, a estrutura dos dados que enviamos de volta.

A View em uma API é o **JSON** (ou XML, etc.) que o cliente (um app mobile, um site) irá receber.

Ela não contém lógica de negócio. Apenas recebe os dados já processados e os formata para entrega. Em frameworks como o Express, a função `res.json(dados)` atua como nosso mecanismo de "geração de View".

Analogia: A View é o prato final montado e servido ao cliente na mesa. Ele é a apresentação do trabalho do Chefe de Cozinha (Model), mas não contém a receita nem os detalhes da cozinha.

O Controller (Controlador): O Maestro da Orquestra

O Controller é o **intermediário**, o ponto de entrada da interação do usuário. Ele atua como um maestro, orquestrando a comunicação entre os outros dois componentes. Suas responsabilidades são:

Receber as requisições do cliente (Ex: `GET /produtos/1`).

Interpretar a requisição e chamar os métodos apropriados do Model para buscar ou manipular dados. (Ex: `ProdutoModel.findById(1)`).

Pegar os dados retornados pelo Model.

Selecionar a View apropriada e enviar os dados para ela, a fim de gerar a resposta final ao cliente. (Ex: `res.status(200).json(produto)`).

Analogia: O Controller é o garçom do "SaborDigital". Ele recebe o pedido do cliente (requisição), leva o pedido para o Chefe de Cozinha (Model), espera o prato ficar pronto, pega o prato pronto (dados) e o leva até a mesa do cliente (resposta). O garçom não cozinha nem come o prato; ele apenas gerencia o fluxo.

Reflita:

No problema da TechFood, onde estava o erro? A lógica de banco de dados (Model), a lógica de requisição (Controller) e a resposta (View) estavam no mesmo lugar. Se o desenvolvedor precisasse apenas alterar a "receita" do produto (Model) para incluir a `categoria`, o processo teria sido muito mais rápido e seguro.

1.2 O Benefício Central: Separação de Preocupações

A principal vantagem do MVC é um princípio de design de software chamado **Separação de Preocupações (Separation of Concerns - SoC)**. Isso significa que cada parte do sistema tem uma única responsabilidade e não interfere nas responsabilidades das outras.

Os impactos no dia a dia de um desenvolvedor são imensos:

Manutenção Facilitada: Precisa mudar uma regra de negócio? Você vai direto ao Model. Precisa mudar como uma rota responde? Você vai ao Controller. Cada alteração é localizada e tem menor risco de quebrar outras partes do sistema.

Reusabilidade: Um mesmo Model `Produto` pode ser usado por vários Controllers para listar produtos, buscar um produto específico ou criar um novo.

Trabalho em Equipe: Um desenvolvedor pode trabalhar na lógica do `UserModel` enquanto outro trabalha no `Auth_Controller`, sem que um pise no trabalho do outro.

Testabilidade: É muito mais fácil escrever testes automatizados para um Model que só lida com dados do que para uma função monolítica que faz tudo ao mesmo tempo.

Dica: *Um bom sinal de uma arquitetura MVC bem aplicada é ter "Fat Models, Skinny Controllers" (Models "gordos", Controllers "magros"). Isso significa que a maior parte da lógica de negócio complexa deve residir nos Models, enquanto os Controllers devem ser apenas uma fina camada de orquestração.*

MÃO NA MASSA

Vamos aplicar esses conceitos à estrutura de pastas e ao fluxo de código do nosso projeto "SaborDigital" usando Node.js e o framework Express.

Passo 1: Estruturando as Pastas do Projeto

Uma estrutura de pastas organizada é o primeiro passo para implementar o MVC. Uma convenção de mercado comum é a seguinte:

#PraCegoVer: Infográfico de diagrama de árvore, estilo flat design. Fundo branco. Representação da estrutura de pastas de um projeto MVC em Node.js. Uma pasta raiz 'sabor-digital-api' se ramifica em 'node_modules' e 'src'. A pasta 'src' se ramifica em três subpastas: 'controllers', 'models' e 'routes'. Ícones de pasta para cada diretório. Dentro de 'controllers', um arquivo com ícone de JavaScript rotulado 'product.controller'. Dentro de 'models', um arquivo 'product.model'. Dentro de 'routes', um arquivo 'product.routes'. Cores sóbrias e profissionais (tons de azul e cinza). Sem texto, apenas rótulos de uma palavra nos arquivos e pastas. Alta legibilidade.

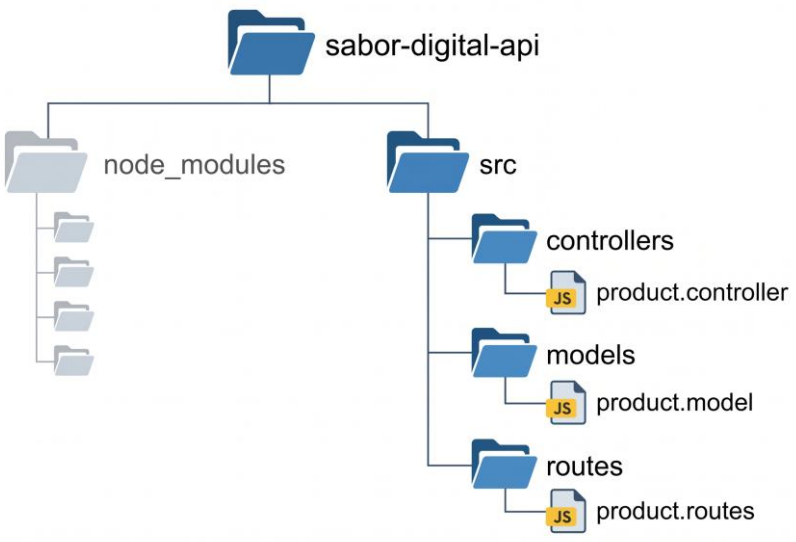

Figura: Gerada por AI

```
/sabor-digital-api
|
|-- /src
|   |-- /controllers          # Onde vivem nossos "maestros"
|   |   |-- product.controller.js
|   |   |-- user.controller.js
|   |
|   |-- /models               # Onde vivem nossos "especialistas"
|   |   |-- product.model.js
|   |   |-- user.model.js
|   |
|   |-- /routes               # O mapa de URLs para os Controllers
|   |   |-- product.routes.js
|   |   |-- user.routes.js
|   |
|   |-- app.js                # Arquivo principal que une tudo
|   |-- server.js             # Arquivo que inicia o servidor
|
|-- package.json
```

Passo 2: O Fluxo na Prática - Listando Produtos

Vamos seguir o caminho de uma requisição `GET /api/products`:

A Rota (`/routes/product.routes.js`): O Ponto de Entrada

O arquivo de rotas é o primeiro a receber a requisição. Ele apenas mapeia a URL e o método HTTP (`GET`) para uma função específica do Controller.

```
    const express = require('express');
    const router = express.Router();
    const productController =
require('../controllers/product.controller');

    // Mapeia GET /api/products para a função 'findAll' do controller
    router.get('/', productController.findAll);

    module.exports = router;
```

O Controller (`/controllers/product.controller.js`): O Maestro

O `productController` recebe o controle. Sua função `findAll` não sabe *como* buscar os produtos, ela apenas delega essa tarefa ao Model.

```
    const Product = require('../models/product.model.js');

    exports.findAll = (req, res) => {
      // O Controller pede ao Model para buscar os dados
      Product.getAll((err, data) => {
        if (err) {
          // Se o Model retornar um erro, o Controller o envia como
resposta
          res.status(500).json({ message: err.message || "Ocorreu um
erro." });
        } else {
          // Se o Model retornar os dados, o Controller os envia como
resposta (a View JSON)
          res.status(200).json(data);
        }
      });
    };
```

Atenção!

Observe que o Controller não contém nenhuma linha de código SQL ou lógica de banco de dados. Ele é apenas um intermediário. Se amanhã trocarmos o

MySQL pelo MongoDB, este arquivo do Controller não precisará de nenhuma alteração!

O Model (`/models/product.model.js`): O Especialista

Finalmente, chegamos ao Model, onde a "mágica" acontece. É aqui que reside a lógica de acesso aos dados.

```
    const sql = require('../config/db.js'); // Conexão com o banco de
dados

    const Product = {}; // Nosso objeto Model

    // O Model implementa o método que o Controller chamou
    Product.getAll = (result) => {
      let query = "SELECT id, nome, descricao, preco, categoria FROM
produtos";

      // Lógica real de acesso ao banco de dados
      sql.query(query, (err, res) => {
        if (err) {
          console.log("error: ", err);
          result(err, null); // Retorna o erro para o callback do
Controller
          return;
        }
        console.log("produtos: ", res);
        result(null, res); // Retorna os dados para o callback do
Controller
      });
    };

    module.exports = Product;
```

Este fluxo demonstra perfeitamente a Separação de Preocupações. Cada arquivo tem um propósito claro e conciso.

RESUMINDO

Ao final deste capítulo, consolidamos os seguintes pontos-chave:

MVC é um padrão de arquitetura que divide a aplicação em três partes: **Model**, **View** e **Controller**.

O **Model** é o cérebro, responsável pelos dados e regras de negócio.

A **View**, em uma API, é a representação dos dados, geralmente em formato **JSON**.

O **Controller** é o maestro, que recebe requisições e coordena a comunicação entre Model e View.

O principal benefício do MVC é a **Separação de Preocupações**, que resulta em um código mais organizado, fácil de manter, escalável e testável.

A estrutura de pastas de um projeto deve refletir essa separação, com diretórios distintos para `controllers`, `models` e `routes`.

Reflita:

Com base no que estudamos, diferencie com suas palavras a responsabilidade de um Model e de um Controller no contexto da nossa API "SaborDigital". O que aconteceria se a query `SELECT * FROM produtos` fosse colocada diretamente dentro do arquivo `product.controller.js`? Qual seria o impacto na manutenção futura do projeto?

PARA CONCLUIR

Parabéns! Você acaba de dar um passo gigantesco em sua formação, saindo do campo de "apenas fazer funcionar" para o de "construir com qualidade profissional". Dominar a arquitetura MVC não é apenas um requisito técnico, é uma mudança de mentalidade que o posicionará como um desenvolvedor mais completo e preparado para os desafios do mercado de trabalho. Nos próximos capítulos, continuaremos a construir e aprimorar a API "SaborDigital" sobre esta base sólida.

Bons estudos e muito sucesso

Capítulo 2: Refatoração para MVC e Design Patterns na Prática

Olá! Seja muito bem-vindo ao segundo capítulo da nossa jornada pela arquitetura de APIs. Com a teoria do MVC em mente, que exploramos anteriormente, é hora de colocar a mão na massa! Nesta unidade, vamos transformar nosso código monolítico em uma aplicação organizada, robusta e pronta para o futuro. Este é o coração da nossa Situação de Aprendizagem (SA2): evoluir nossa API, aplicando um padrão que é referência de mercado e essencial para qualquer desenvolvedor back-end.

Durante nossos estudos, vamos explorar como refatorar uma aplicação real, indo além do MVC básico para conhecer padrões de projeto complementares que elevam a qualidade do nosso software. Ao final desta jornada, você terá uma compreensão profunda de como estruturar uma API de forma profissional, facilitando a manutenção, a testabilidade e a evolução do projeto. Você será capaz de aplicar os padrões de Camada de Serviço e Repositório, habilidades altamente valorizadas no mercado de trabalho.

DESAFIO

Confira, a seguir, uma situação prática relacionada aos conteúdos que vamos estudar.

A startup **TechFood**, para a qual estamos desenvolvendo a API **"SaborDigital"**, está crescendo rapidamente. Com o sucesso inicial, novos desenvolvedores estão se juntando à equipe. O gerente de projetos, observando o código do endpoint `POST /produtos`, percebeu um problema: toda a lógica — validação da requisição, regras de negócio e inserção no banco de dados — está misturada em um único arquivo, um

grande bloco de código. Isso torna a manutenção difícil e a curva de aprendizado para os novos membros da equipe, muito alta.

Ele deu o sinal verde para a refatoração. Seu desafio é pegar o endpoint `POST /produtos` e reestruturá-lo completamente, seguindo o padrão MVC. A tarefa consiste em criar o arquivo `ProductController.js` e o `ProductModel.js` (ou `ProductRepository.js`), mover a lógica de validação e de interação com o banco de dados para seus devidos lugares, garantindo que, ao final, o endpoint continue funcionando exatamente como antes, mas com seu código interno totalmente reorganizado, limpo e escalável.

ESTUDO E PRÁTICA

2.1. A Arquitetura MVC como Pilar da Organização

Como vimos, a sigla MVC significa **Model-View-Controller**. Em APIs REST, que não possuem uma "View" tradicional (como uma página HTML renderizada no servidor), costumamos adaptar o conceito. O "V" de View é substituído pelo cliente que consome a API (um aplicativo mobile, um front-end em React, etc.). O fluxo se torna:

Requisição do Cliente: Um aplicativo faz uma requisição `POST` para `/produtos`.

Controller: A rota aponta para um método no **Controller**. Ele é o "maestro" da operação. Sua função é receber a requisição HTTP, extrair os dados relevantes (como o `body`) e orquestrar os próximos passos. Ele não deve conter regras de negócio ou acesso direto ao banco de dados.

Model: O **Controller** invoca o **Model**, que é responsável por representar os dados e interagir com o banco de dados. É aqui que a

lógica de "salvar um produto", "buscar um produto" ou "atualizar um produto" reside.

Imagine um restaurante de alta gastronomia: o **Controller** é o garçom. Ele recebe seu pedido (`POST /produtos`), anota os detalhes (`{ "nome": "Pizza", "preco": 50 }`), mas não vai para a cozinha preparar. Ele apenas entrega a comanda para a cozinha (**Model**), que sabe exatamente como preparar o prato e onde buscar os ingredientes (banco de dados).

#PraCegoVer: Diagrama técnico isométrico, estilo flat design moderno. Fundo branco limpo. Representação visual da arquitetura MVC para APIs. Três blocos 3D distintos e conectados: 'Model' com um ícone de banco de dados, 'Controller' com um ícone de torre de controle ou cérebro, e 'Client' (substituindo a View) com um ícone de smartphone. Setas nítidas indicam o fluxo: Client -> Controller -> Model -> Controller -> Client. Cores institucionais sóbrias (azul marinho, cinza, branco). Sem texto, apenas rótulos de uma palavra em cada bloco: 'Client', 'Controller', 'Model'. Alta definição.

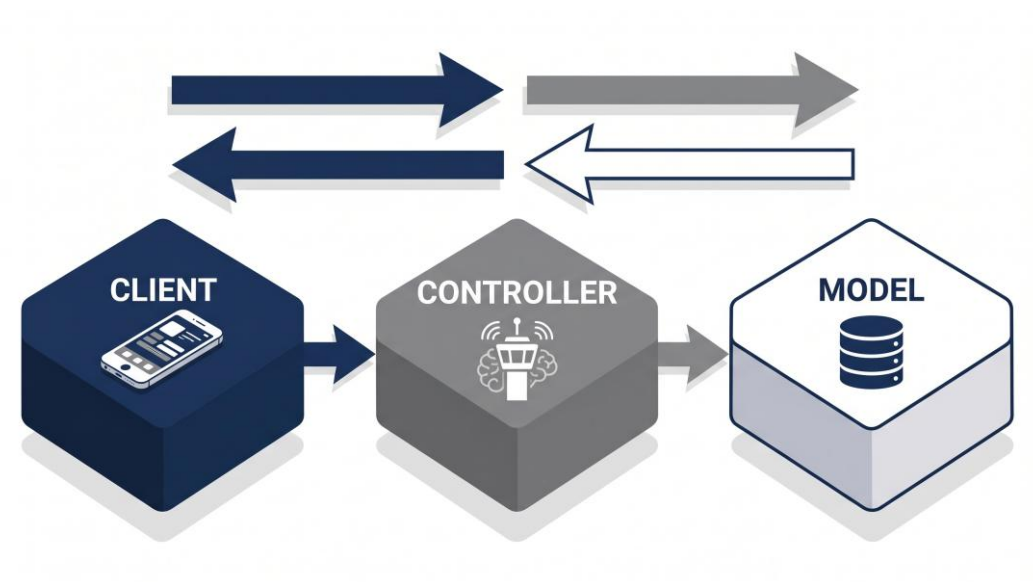

Figura: Gerada por AI

Atenção! *Um erro comum é colocar lógica de negócio dentro do Controller. Se o Controller precisa verificar se um e-mail já existe ou se o preço de um produto*

está dentro de uma faixa permitida, ele está fazendo mais do que deveria. Sua única responsabilidade é gerenciar o fluxo da requisição e da resposta HTTP.

2.2. Indo Além: Padrões de Projeto para um Código Blindado

Apenas separar em Model e Controller já é um grande avanço, mas podemos ir além. No desenvolvimento de software profissional, usamos padrões de projeto (Design Patterns) para resolver problemas recorrentes. Para nossa API, dois padrões são especialmente úteis: a **Camada de Serviço (Service Layer)** e o **Padrão de Repositório (Repository Pattern)**.

Camada de Serviço (Service Layer)

O que acontece quando a "regra de negócio" é complexa? Por exemplo, ao criar um produto, talvez precisemos:

Verificar se o nome já existe.
Calcular um imposto com base na categoria.
Enviar um e-mail de notificação para o estoque.

Colocar tudo isso no Controller o deixaria inchado. Colocar no Model misturaria a lógica de negócio com a lógica de acesso a dados. A solução é a **Camada de Serviço**.

O **Service** fica entre o Controller e o Model/Repository. Ele contém toda a lógica de negócio pura.

Controller: Recebe a requisição, chama o Service.
Service: Executa as regras de negócio (validações, cálculos, orquestração de múltiplas ações).
Model/Repository: É chamado pelo Service para persistir os dados.

Dica: *Pense no Service como o* ***Chef Executivo*** *da cozinha. O garçom (Controller) entrega a comanda. O Chef (Service) não suja as mãos com os*

ingredientes, mas ele coordena os cozinheiros (Repositories) e garante que o prato (regra de negócio) seja executado com perfeição.

Padrão de Repositório (Repository Pattern)

O Padrão de Repositório tem um objetivo claro: **abstrair a camada de acesso a dados**. Em vez de o nosso Service (ou Model) escrever `db.query('INSERT INTO ...')` ou `prisma.product.create(...)`, ele simplesmente chama um método como `productRepository.create(productData)`.

O Repositório é uma classe cujo único trabalho é se comunicar com o banco de dados. Isso traz uma vantagem gigantesca: se um dia a TechFood decidir migrar do banco de dados MySQL para um banco NoSQL como o MongoDB, precisaríamos alterar apenas a implementação dentro do `ProductRepository`. O resto da aplicação (`Service`, `Controller`) continuaria intacto, pois eles não sabem (e não precisam saber) qual banco de dados está sendo usado.

#PraCegoVer: Diagrama de fluxo de camadas em estilo flat design corporativo. Fundo cinza claro. Cinco camadas horizontais representando o fluxo de uma requisição em uma API moderna. De cima para baixo: 1. 'Route' (ícone de mapa de estrada), 2. 'Controller' (ícone de torre de controle), 3. 'Service' (ícone de cérebro ou placa de circuito), 4. 'Repository' (ícone de cofre ou estante), 5. 'Database' (ícone de cilindro de banco de dados). Setas verticais nítidas conectam as camadas, mostrando o fluxo descendente da requisição e o fluxo ascendente da resposta. Cores: azul para fluxo, cinza para camadas. Sem texto, apenas rótulos de uma palavra.

Figura: Gerada por AI

Reflita: *Com essa nova arquitetura (Controller -> Service -> Repository), a responsabilidade de cada arquivo fica extremamente clara. Isso não apenas facilita a manutenção, mas também a criação de testes automatizados. Poderíamos testar uma regra de negócio no Service sem precisar de um banco de dados real, por exemplo. Como isso impacta a velocidade e a confiabilidade do desenvolvimento?*

MÃO NA MASSA

Vamos aplicar essa estrutura para refatorar o endpoint `POST /produtos` da nossa API "SaborDigital".

Situação Inicial (Antes): Um único arquivo `index.js` com todo o código.

```
// index.js (ANTES)
app.post('/produtos', async (req, res) => {
  const { nome, preco, categoria } = req.body;

  // 1. Validação misturada
  if (!nome || !preco) {
    return res.status(400).json({ message: 'Nome e preço são
```

```
obrigatórios.' });
  }

  // 2. Regra de negócio misturada
  if (preco < 0) {
    return res.status(400).json({ message: 'Preço não pode ser negativo.'
});
  }

  try {
    // 3. Acesso direto ao banco de dados misturado
    const [result] = await db.promise().query(
      'INSERT INTO produtos (nome, preco, categoria) VALUES (?, ?, ?)',
      [nome, preco, categoria]
    );
    res.status(201).json({ id: result.insertId, nome, preco, categoria });
  } catch (error) {
    res.status(500).json({ message: 'Erro ao criar produto.' });
  }
});
```

Passo 1: Criar a Estrutura de Diretórios

Dentro da pasta do seu projeto (ex: `src/`), crie a seguinte estrutura:

```
src/
├── controllers/
├── models/
├── repositories/
├── services/
└── routes/
```

Passo 2: Criar a Rota e o Controller

Primeiro, isolamos a definição da rota.

```
// src/routes/productRoutes.js
const express = require('express');
const router = express.Router();
const ProductController = require('../controllers/ProductController');

// A rota agora apenas aponta para o método do controller
router.post('/produtos', ProductController.create);
```

```
module.exports = router;
```

Agora, o `ProductController`. Sua função é apenas lidar com `req` e `res`.

```
// src/controllers/ProductController.js
const ProductService = require('../services/ProductService');

class ProductController {
  async create(req, res) {
    try {
      const { nome, preco, categoria } = req.body;
      const produto = await ProductService.create(nome, preco, categoria);
      return res.status(201).json(produto);
    } catch (error) {
      // O service vai gerar erros específicos que podemos tratar
      return res.status(error.statusCode || 500).json({ message:
error.message });
    }
  }
}

module.exports = new ProductController();
```

Passo 3: Criar o Repositório

Este arquivo terá a única responsabilidade de falar com o banco de dados.

```
// src/repositories/ProductRepository.js
const db = require('../config/database'); // Supondo que você tenha a
conexão do DB aqui

class ProductRepository {
  async create(productData) {
    const { nome, preco, categoria } = productData;
    const [result] = await db.promise().query(
      'INSERT INTO produtos (nome, preco, categoria) VALUES (?, ?, ?)',
      [nome, preco, categoria]
    );
    return { id: result.insertId, ...productData };
  }
}
```

```
module.exports = new ProductRepository();
```

Passo 4: Criar o Serviço (O Cérebro da Operação)

Aqui implementaremos as validações e regras de negócio.

```
// src/services/ProductService.js
const ProductRepository = require('../repositories/ProductRepository');

class ProductService {
  async create(nome, preco, categoria) {
    // 1. Validações e Regras de Negócio
    if (!nome || !preco) {
      const error = new Error('Nome e preço são obrigatórios.');
      error.statusCode = 400; // Bad Request
      throw error;
    }

    if (preco < 0) {
      const error = new Error('Preço não pode ser negativo.');
      error.statusCode = 400; // Bad Request
      throw error;
    }

    const productData = { nome, preco, categoria };

    // 2. Chama o repositório para persistir os dados
    const newProduct = await ProductRepository.create(productData);

    // Podemos fazer mais coisas aqui, como enviar um e-mail, etc.

    return newProduct;
  }
}

module.exports = new ProductService();
```

Finalmente, no seu arquivo principal (`index.js` ou `app.js`), você apenas registra as rotas:

```
// index.js (DEPOIS)
const express = require('express');
const app = express();
const productRoutes = require('./src/routes/productRoutes');

app.use(express.json());
app.use('/api', productRoutes); // Prefixo /api para as rotas

// ... resto da configuração do servidor
```

Observe como o código agora está limpo, organizado e cada arquivo tem uma responsabilidade única (Princípio da Responsabilidade Única - SOLID).

RESUMINDO

Para consolidar o que aprendemos, vamos revisar os pontos-chave:

Refatoração é o processo de reestruturar o código internamente sem alterar seu comportamento externo.

A arquitetura **MVC** (adaptada para APIs) separa as responsabilidades em: **Controller** (gerencia o fluxo HTTP), **Model/Repository** (acessa os dados) e o **Client** (consome a API).

A **Camada de Serviço (Service Layer)** é introduzida para abrigar as **regras de negócio**, mantendo Controllers e Repositories enxutos e focados.

O **Padrão de Repositório (Repository Pattern)** abstrai o acesso ao banco de dados, tornando a aplicação mais flexível e fácil de manter ou migrar.

Ganhos da Nova Arquitetura:

Legibilidade: É fácil entender o que cada arquivo faz.

Manutenibilidade: Alterar uma regra de negócio não exige modificar o Controller ou o acesso ao banco de dados.

Testabilidade: Cada camada (Service, Repository) pode ser testada de forma isolada.

Checkpoint de Conhecimento: Em nossa nova arquitetura, se precisarmos adicionar uma validação que impede o cadastro de produtos com nomes duplicados, em qual camada (Controller, Service ou Repository) essa lógica deveria ser implementada? Por quê?

Resposta: Na camada de **Serviço (Service)**.

Por quê? A verificação de um nome duplicado é uma **regra de negócio** da aplicação. Não é uma responsabilidade do Controller (que lida com HTTP) nem puramente do Repository (que apenas executa comandos no banco). O Service orquestraria a ação: antes de chamar `ProductRepository.create()`, ele chamaria um método como `ProductRepository.findByName()` para verificar a existência do produto. A decisão de prosseguir ou lançar um erro é lógica de negócio, pertencente ao Service.

PARA CONCLUIR

Parabéns! Você acaba de dar um passo fundamental na sua evolução como desenvolvedor back-end. Transformar um código monolítico em uma arquitetura de camadas bem definida não é apenas uma "boa prática", é uma necessidade no mundo real para construir sistemas que duram e se adaptam. A habilidade de organizar o código de forma lógica e escalável é o que diferencia um programador júnior de um sênior.

Continue praticando essa estrutura em seus projetos. No início, pode parecer mais trabalhoso, mas os benefícios em clareza, colaboração e manutenção se pagarão exponencialmente ao longo do tempo.

Bons estudos e muito sucesso

SENAI

Apostila criada por: Celso Rodrigo Giusti